

C++基础

指针

```
#include<iostream>
using namespace std;
int main()
{
    int a,b;
    int *p=&a;//定义指针p，指针p指向a的地址

    //int *p1=NULL;定义空指针
    // *p1=100;空指针不可以进行访问，因为0~255之间的内存编号是系统占用的，因此不可以访问
    a=100;
    cout<<p<<endl; //输出a的地址
    cout<<*p<<endl; //*：代表解引用；输出100
    cout<<sizeof(int *)<<endl;
    cout<<sizeof(float *)<<endl;
    cout<<sizeof(double *)<<endl;
    cout<<sizeof(char *)<<endl; //64位操作系统下，指针占8个字节
    return 0;
}
```

const指针

```
#include<iostream>
using namespace std;
int main()
{
    int a=10;

    //常量指针
    const int *p1=&a;//指针p的指向可以改，指针p指向的值不可以改
    cout<<*p1<<endl;

    //指针常量
    int * const p2=&a;//指针p2的指向不可以改，指针p指向的值可以更改
    cout<<*p2<<endl;

    //const修饰常量和指针
    const int * const p3=&a;//指针p3的指向不可以更改，指针p指向的值也不可以更改
    cout<<*p3<<endl;
}
```

指针与函数

```
#include<iostream>
using namespace std;

//地址传递
void swap(int *a,int *b)
{
    int temp=*a;
    *a=*b;
    *b=temp;
}

int main()
{
    int a=10;
    int b=20;
    swap(&a,&b); //函数中的形参定义*a与*b两个指针指向两个地址，所以调用函数时传入的参数是地址
    cout<<a<<endl; //20
    cout<<b<<endl; //10
}
```

指针与数组

```
#include<iostream>
using namespace std;
int main()
{
    int arr[10]={1,2,3,4,5,6,7,8,9,10};
    int *p=arr; //数组名就是数组的首地址
    cout<<*p<<endl;

    for(int i=0;i<=9;i++)
    {
        cout<<*p<<endl;
        p++; //分配连续的空间，将指针向后移4B，指针p就指向了数组中第二个元素
    }
}
```

案例-指针函数数组实现冒泡排序

```
#include<iostream>
using namespace std;

void bubblesort(int *arr, int len)
{
    for (int i = 0; i < len - 1; i++) // 外层循环控制排序轮数
    {
```

```

        for (int j = 0; j < len - i - 1; j++) // 内层循环控制每轮比较次数
        {
            if (arr[j] > arr[j + 1]) // 如果前一个元素大于后一个元素，交换它们
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void printarray(int *arr,int len)
{
    for(int i=0;i<len;i++)
    {
        cout<<arr[i]<<endl;
    }
}

int main()
{
    int arr[10]={3,5,9,1,4,7,2,0,8,6};
    int len=sizeof(arr)/sizeof(arr[0]); //sizeof(arr)为数组arr[10]所占的内存空间大小
    bubblesort(arr,len); //arr为数组首地址，所以函数形参定义一个指针int *arr;
    printarray(arr,len);
    return 0;
}

```

结构体

```

#include<iostream>
using namespace std;

struct student//定义结构体：一些类型的集合组成的一个类型
{
    string name;
    int age;
    int score;
}; //一定要加“;” //创建方式3 ;之后加结构体变量名s3

int main()
{
    //创建方式1
    struct student s1; //创建结构体变量时，struct可以省略
    s1.name="zq";
    s1.age=22;
    s1.score=350;
    cout<<s1.name<<" "<<s1.age<<" "<<s1.score<<endl;

    //创建方式2
    struct student s2={"zrq",22,320};
    cout<<s2.name<<" "<<s2.age<<" "<<s2.score<<endl;
}

```

```
    return 0;
}
```

结构体数组

```
#include<iostream>
#include<string>
using namespace std;

struct student
{
    string name;
    int age;
    int score;
};

int main()
{
    struct student arr[3]=
    {
        {"zq",22,350},
        {"zrq",22,320},
        {"zz",22,380}
    };

    arr[2].name="csh";
    arr[2].age=22;
    arr[2].score=320;

    for(int i=0;i<3;i++)
    {
        cout<<arr[i].name<<" ";
        cout<<arr[i].age<<" ";
        cout<<arr[i].score<<endl;
    }
    return 0;
}
```

结构体指针

```
#include<iostream>
#include<string>
using namespace std;

struct student
{
    string name;
    int age;
    int score;
};
```

```

int main()
{
    struct student s={"zrq",22,320};
    struct student *p=&s;//struct可以省略
    //通过结构体指针访问结构体中的属性，需要用“->”
    cout<<p->name<<" "<<p->age<<" "<<p->score<<endl;
    cout<<*p;
    return 0;
}

```

嵌套结构体

```

#include<iostream>
#include<string>
using namespace std;

struct student
{
    string name;
    int age;
    int score;
};

struct teacher
{
    int id;
    string name;
    int age;
    struct student s;
};

int main()
{
    struct teacher t;
    t.id=320705;
    t.age=30;
    t.name="zq";
    t.s.name="zrq";
    t.s.age=18;
    t.s.score=150;

    cout<<t.id<<" "<<t.name<<" "<<t.age<<" "<<endl;
    cout<<t.s.name<<" "<<t.s.age<<" "<<t.s.score<<endl;
    return 0;
}

```

结构体做函数参数

```
#include<iostream>
#include<string>
using namespace std;

struct student
{
    string name;
    int age;
    int score;
};

//值传递
void printstudent1(struct student s)
{
    s.age=33;
    cout<<s.name<<" "<<s.age<<" "<<s.score<<endl;
}

//地址传递
void printstudent2(struct student *p)
{
    p->age=11;
    cout<<p->name<<" "<<p->age<<" "<<p->score<<endl;
}

int main()
{
    struct student s;
    s.name="zq";
    s.age=22;
    s.score=350;

    printstudent1(s);
    cout<<s.name<<" "<<s.age<<" "<<s.score<<endl; //值传递不改变实参

    printstudent2(&s);
    cout<<s.name<<" "<<s.age<<" "<<s.score<<endl; //地址传递改变实参

    return 0;
}
```

结构体中const

```
#include<iostream>
using namespace std;

struct student
{
    string name;
    int age;
```

```
    int score;
};

void printstudent(const struct student *p)
{
    //p.age=10; 加上const防止误操作
    cout<<p->name<<" "<<p->age<<" "<<p->score<<endl;
}

int main()
{
    struct student s={"zq",22,350};

    printstudent(&s);

    return 0;
}
```